



University Institute of Engineering
Department of Computer Science & Engineering

Experiment-1

Student Name: Pranav Chuahan
Branch: BE CSE (cyber security)
Semester: 5th
Subject Name: ADBMS

UID: 24BCY70004
Section/Group: 24AIT_KRG-G2
Date of Performance: 11/07/2026
Subject Code: 24CSP-333

1. Aim of the practical: To study and implement advanced SQL queries using conditional logic, aggregate functions, joins, self-joins, and date operations to solve real-world database problems

2. Questions

(A) Write an SQL solution to calculate the total number of bank accounts belonging to each salary category. The classification categories are:

- "Low Salary":** All monthly salaries strictly less than \$20,000.
- "Average Salary":** All monthly salaries in the inclusive range [\$20,000, \$50,000].
- "High Salary":** All monthly salaries strictly greater than \$50,000. The final result table must report all three categories, even if a category contains zero accounts.

```
-- Question 1
CREATE TABLE accounts (
    account_id INT PRIMARY KEY,
    income INT
);

INSERT INTO accounts (account_id, income)
VALUES
(2, 12747),
(3, 108939),
(6, 91738),
(7, 45169),
(8, 87709);

SELECT * FROM accounts;
```

	account_id [PK] integer	income integer
1	2	12747
2	3	108939
3	6	91738
4	7	45169
5	8	87709



University Institute of Engineering

Department of Computer Science & Engineering

Solution:

```
SELECT 'LOW SALARY' AS CATEGORY, COUNT(ACCOUNT_ID) AS ACCOUNT_COUNT
FROM ACCOUNTS
WHERE INCOME < 20000

UNION ALL

SELECT 'AVERAGE SALARY' AS CATEGORY, COUNT(ACCOUNT_ID) AS ACCOUNT_COUNT
FROM ACCOUNTS
WHERE INCOME >= 20000 AND INCOME <= 50000

UNION ALL

SELECT 'HIGH SALARY' AS CATEGORY, COUNT(ACCOUNT_ID) AS ACCOUNT_COUNT
FROM ACCOUNTS
WHERE INCOME >50000;
```

	category text	account_count bigint
1	LOW SALARY	1
2	AVERAGE SALA...	1
3	HIGH SALARY	3

(B) Design an Employee Management System database architecture by implementing the following operations sequentially:

1. Create and populate structural tracking tables for Departments, Employees, and Salaries.
2. Query the system using multi-table Joins to pull clear department breakdowns, apply a 10% salary enhancement across all members of the HR department, and delete any individual salary record dropping strictly below \$30,000.
3. List all remaining active employees whose personal salary exceeds the overall company-wide average salary.



University Institute of Engineering

Department of Computer Science & Engineering

```
-- Question 2
CREATE TABLE Departments (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50) NOT NULL
);

CREATE TABLE Employees (
    emp_id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES Departments(dept_id)
);

CREATE TABLE Salaries (
    emp_id INT PRIMARY KEY,
    salary INT NOT NULL,
    FOREIGN KEY (emp_id)
        REFERENCES Employees(emp_id)
        ON DELETE CASCADE
);
```

```
INSERT INTO Departments (dept_id, dept_name)
VALUES (10, 'HR'), (20, 'IT'), (30, 'Sales');

INSERT INTO Employees (emp_id, name, dept_id)
VALUES (1, 'Alice', 10), (2, 'Bob', 10), (3, 'Charlie', 20),
(4, 'David', 20), (5, 'Emma', 30);

INSERT INTO Salaries (emp_id, salary)
VALUES (1, 50000), (2, 25000), (3, 80000), (4, 40000), (5, 45000);
```

Solution:

1.

```
--part (1)
SELECT E.*, D.*, S.*
FROM Employees AS E
JOIN Departments AS D
ON E.dept_id = D.dept_id
JOIN Salaries AS S
ON E.emp_id = S.emp_id;
```

	emp_id integer	name character va	dept_id integer	dept_id integer	dept_name character va	emp_id integer	salary integer
1	1	Alice	10	10	HR	1	50000
2	2	Bob	10	10	HR	2	25000
3	3	Charlie	20	20	IT	3	80000
4	4	David	20	20	IT	4	40000
5	5	Emma	30	30	Sales	5	45000



University Institute of Engineering

Department of Computer Science & Engineering

2.

```
--part (2)
UPDATE Salaries
SET salary = salary + salary * 0.10
WHERE emp_id IN (
    SELECT E.emp_id
    FROM Employees AS E
    JOIN Departments AS D
    ON E.dept_id = D.dept_id
    WHERE D.dept_name = 'HR'
);
```

Before

	emp_id [PK] integer	salary integer
1	1	50000
2	2	25000
3	3	80000
4	4	40000
5	5	45000

After

	emp_id [PK] integer	salary integer
1	3	80000
2	4	40000
3	5	45000
4	1	55000
5	2	27500

3.

```
--part (3)
SELECT E.emp_id
FROM Employees AS E
JOIN Salaries AS S
ON E.emp_id = S.emp_id
WHERE S.salary > (SELECT AVG(salary) FROM Salaries);
```

	emp_id [PK] integer
1	3
2	1



University Institute of Engineering
Department of Computer Science & Engineering

(C) A pizza restaurant stores all the available pizza toppings and their ingredient costs in a table called `pizza_toppings`.

Every customer must choose exactly 3 different toppings to prepare a pizza.

Write a PostgreSQL query to generate every possible 3-topping pizza combination along with its total ingredient cost.

Requirements

- a. Every pizza must contain exactly 3 different toppings.
- b. The same topping cannot appear more than once in a pizza.
- c. Different arrangements of the same toppings should be considered the same pizza.

Example:

- i. Chicken, Pepperoni, Sausage
- ii. Pepperoni, Chicken, Sausage
- iii. Sausage, Chicken, Pepperoni

These are all the same pizza, so only one should appear.

Display the topping names separated by commas.

Display the total ingredient cost of each pizza.

Sort the output:

- a. First by highest total cost
- b. If two pizzas have the same cost, sort them alphabetically.



University Institute of Engineering

Department of Computer Science & Engineering

```
-- Question 3
CREATE TABLE pizza_toppings (
    topping_name VARCHAR(50) PRIMARY KEY,
    ingredient_cost NUMERIC(4,2) NOT NULL
);

INSERT INTO pizza_toppings (topping_name, ingredient_cost) VALUES
('Pepperoni', 0.50),
('Sausage', 0.70),
('Chicken', 0.55),
('Onions', 0.25),
('Extra Cheese', 0.40);
```

Solution:

```
SELECT
    CONCAT(
        T1.topping_name, ', ',
        T2.topping_name, ', ',
        T3.topping_name
    ) AS pizza,
    (T1.ingredient_cost +
     T2.ingredient_cost +
     T3.ingredient_cost) AS total_cost
FROM pizza_toppings AS T1
JOIN pizza_toppings AS T2
ON T1.topping_name < T2.topping_name
JOIN pizza_toppings AS T3
ON T2.topping_name < T3.topping_name
ORDER BY
    total_cost DESC,
    pizza ASC;
```

	pizza text	total_cost numeric
1	Chicken, Pepperoni, Sausage	1.75
2	Chicken, Extra Cheese, Sausage	1.65
3	Extra Cheese, Pepperoni, Saus...	1.60
4	Chicken, Onions, Sausage	1.50
5	Chicken, Extra Cheese, Pepper...	1.45
6	Onions, Pepperoni, Sausage	1.45
7	Extra Cheese, Onions, Sausage	1.35
8	Chicken, Onions, Pepperoni	1.30
9	Chicken, Extra Cheese, Onions	1.20
10	Extra Cheese, Onions, Pepperoni	1.15



University Institute of Engineering
Department of Computer Science & Engineering

(D) Amazon wants to identify returning customers who made another purchase shortly after their first purchase.

Write a PostgreSQL query to find all users who made another purchase within 1 to 7 days of an earlier purchase.

Requirements

- a. Compare purchases made by the same user only.
- b. Ignore comparisons where the same transaction is matched with itself.
- c. The second purchase must occur after the first purchase.
- d. Same-day purchases must not be considered.
- e. The second purchase must occur within 7 days of the first purchase.
- f. Display only the unique User IDs that satisfy the above conditions.
- g. Sort the output in ascending order of user_id.

```
-- Question 4
CREATE TABLE amazon_transactions (
    id INT PRIMARY KEY,
    user_id INT NOT NULL,
    item VARCHAR(50),
    created_at TIMESTAMP NOT NULL,
    revenue INT
);

INSERT INTO amazon_transactions (id, user_id, item, created_at, revenue) VALUES
(1, 101, 'biscuit', '2026-03-01 10:00:00', 12),
(2, 101, 'milk', '2026-03-01 14:00:00', 5),
(3, 101, 'bread', '2026-03-05 09:00:00', 8),
(4, 102, 'banana', '2026-03-10 12:00:00', 4),
(5, 102, 'apple', '2026-03-25 11:00:00', 6),
(6, 103, 'milk', '2026-03-15 08:00:00', 5),
```



University Institute of Engineering
Department of Computer Science & Engineering

Solution:

```
SELECT DISTINCT A.user_id
FROM amazon_transactions A
JOIN amazon_transactions B
ON A.user_id = B.user_id
WHERE B.created_at > A.created_at
      AND DATE(B.created_at) > DATE(A.created_at)
      AND B.created_at <= A.created_at + INTERVAL '7 days'
ORDER BY A.user_id;
```

	user_id 
1	101
2	103

3. Learning Outcomes: Install and configure MySQL Server and MySQL Workbench.

- Learned to create and manage database tables using PostgreSQL DDL commands.
- Performed data retrieval and manipulation using SQL queries, joins, and subqueries.
- Applied aggregate functions, conditional statements, and sorting to analyze data.
- Implemented self-joins to solve real-world combination and transaction-based problems.
- Developed practical skills in writing efficient SQL queries for database applications.

Evaluation Grid:

Sr. No.	Parameters	Marks Obtained	Maximum Marks
1.	Student Performance (Conduct of experiment)		12
2.	Viva Voce		10
3.	Submission of Work Sheet (Record)		8
	Signature of Faculty (with Date):	Total Marks Obtained:	30